

Open table formats

Apache Iceberg, Amazon S3 Tables, and the row-level operations a plain data lake cannot do.

What the metadata layer looks like

01

Catalog

Points at the current metadata file for the table.

02

Metadata file

Schema, partitioning, and the list of snapshots.

03

Manifest list

One per snapshot. Names the manifests in it.

04

Manifest

Lists data files, with per-column min and max values.

05

Data files

The actual Parquet. Never modified in place.

ON THE EXAM

A commit swaps one pointer in the catalog. That single atomic change is where the transaction guarantee comes from.

A pile of Parquet files in S3 is not a table. It is **files that happen to have the same columns.**

What a plain lake cannot do

- ◆ **Update or delete a single row.** Objects are immutable, so you rewrite whole files.
- ◆ **Guarantee a consistent read.** A reader can see a half-finished write.
- ◆ **Change a schema safely.** Adding or renaming a column can break existing readers.
- ◆ **Look at yesterday's version.** Once overwritten, the old state is gone.
- ◆ **Let two writers work at once** without one silently overwriting the other.

ON THE EXAM

Every one of these is a problem this course has already described and deferred. This is where they get solved.

Iceberg adds a **metadata layer** that says exactly which files make up the table, right now.

What that buys you

- ◆ **ACID transactions.** A write is visible completely or not at all.
- ◆ **Row-level update, delete and merge.** Expressed as SQL, not as file rewriting.
- ◆ **Time travel.** Query the table as it stood at a timestamp or a snapshot.
- ◆ **Schema evolution.** Add, drop, rename and reorder columns safely.
- ◆ **Partition evolution.** Change the partitioning without rewriting history.
- ◆ **Concurrent writers,** resolved by optimistic concurrency rather than by luck.

ON THE EXAM

Partition evolution is the one people find most surprising. Old data keeps its old layout and still queries correctly.

How a delete is actually written

COPY ON WRITE

Rewrite the file

- Affected files rewritten without those rows
- Writes slow, reads fast
- No merging needed at query time
- Suits infrequent updates, read-heavy tables

MERGE ON READ

Record the change

- A delete file records which rows to ignore
- Writes fast, reads do more work
- Compaction later folds them in
- Suits frequent updates and streaming

THE TIEBREAKER

Frequent small updates favour merge-on-read. Occasional bulk changes on a heavily queried table favour copy-on-write.

Hidden partitioning

- ◆ **In a plain lake**, a query must filter on the partition column to prune.
- ◆ Filter on a timestamp when you partitioned on a date string, and nothing prunes.
- ◆ **Iceberg records the transform**, such as the day of a timestamp column.
- ◆ A query filtering that timestamp prunes correctly without knowing the layout.
- ◆ **Transforms include** year, month, day, hour, bucket and truncate.

ON THE EXAM

Hidden partitioning removes a whole class of silent query mistake, where a filter looks right and prunes nothing.

How schema evolution actually works

- ◆ **Columns have IDs**, not just names. A rename changes the name, not the identity.
- ◆ **Add a column** and old files simply return null for it.
- ◆ **Drop a column** and nothing is rewritten. It stops being projected.
- ◆ **In a plain Parquet lake** a rename means two incompatible sets of files.
- ◆ **Contrast with Avro**, which handles evolution well but is row based.

ON THE EXAM

Column IDs are why a rename is safe. Name-based formats have to guess whether a renamed column is a new one.

Time travel, and what it is for

- ◆ **Every write creates a snapshot.** The metadata retains previous ones.
- ◆ **Query as of a timestamp or snapshot ID** to see the table as it was.
- ◆ **Roll back** to a previous snapshot after a bad write.
- ◆ **Reproducibility.** Re-run a report against the exact data it originally used.
- ◆ **Snapshots cost storage.** Expiring them is a maintenance task, not optional.

ON THE EXAM

A bad job that corrupted a table is a rollback, not a restore from backup. That is a genuinely different answer.

Branches and tags

- ◆ A **tag** names a snapshot so it survives expiry. End of month, for instance.
- ◆ A **branch** is an independent line of commits on the same table.
- ◆ **Write, audit, publish.** Write to a branch, validate it, then merge to main.
- ◆ Consumers never see data that failed validation, because it never reached main.
- ◆ Retention can be set per branch and per tag.

ON THE EXAM

Write-audit-publish is how quality checks stop bad data reaching consumers, rather than reporting it afterwards.

Asking the table about itself

- ◆ **Metadata tables are queryable with SQL.** Snapshots, history, files, manifests.
- ◆ **Query the snapshots table** to find the snapshot ID from before a bad job.
- ◆ **Query the files table** to see file sizes and prove a small-file problem.
- ◆ **The history table** shows which snapshot was current at any moment.
- ◆ Works in Athena, and in Spark on EMR and Glue.

ON THE EXAM

This is how you find the snapshot to roll back to. It is not guesswork, it is a query.

Iceberg, Hudi or Delta Lake?

APACHE ICEBERG

The AWS default

- Named explicitly in the exam guide
- Native in S3 Tables, Glue, Athena, Redshift, EMR
- Strong schema and partition evolution
- The safe answer when the question does not specify

HUDI AND DELTA LAKE

Also supported

- Both work on Glue and EMR
- Hudi is strong at upserts and incremental pulls
- Delta comes from the Databricks ecosystem
- Choose them when the question names them

THE TIEBREAKER

AWS has standardised on Iceberg. Unless a question names Hudi or Delta, Iceberg is the intended answer.

A healthcare company stores ten years of patient interaction records as Parquet files in Amazon S3, partitioned by month, queried with Amazon Athena. Under privacy legislation the company must erase all records for a given patient within thirty days of a request, and receives around forty such requests a month.

Currently an engineer identifies every affected file, rewrites each one without the patient's rows, and replaces it. The process takes two days per request and has twice produced corrupted partitions when a rewrite failed midway.

What should the company implement?

- ☐ A Convert the dataset to Apache Iceberg tables and use DELETE statements to remove the rows
- ☐ B Partition the data by patient identifier so entire files can be deleted per request
- ☐ C Enable S3 Versioning so a failed rewrite can be rolled back to the previous version
- ☐ D Move the data to Amazon Redshift, where DELETE statements are supported

◆ PAUSE AND DECIDE

What should the company implement?

- A

 Convert to Iceberg, use DELETE
Row-level deletion becomes one SQL statement, applied atomically. A failure leaves the table untouched rather than corrupted.
- B

 Partition by patient ID
Module eight's over-partitioning trap. Millions of tiny partitions, and queries filter by date, not by patient.
- C

 Enable versioning
Makes the corruption recoverable without making the process any less manual or any faster. Two days per request remains.
- D

 Move to Redshift
Solves deletion by abandoning the lake and loading ten years into a warehouse. Enormous change for one requirement.

THE SIGNAL

Row-level deletion inside shared files is the exact problem open table formats exist to solve.

Every write creates files and snapshots.
Without maintenance, an Iceberg table
gets **slower over time**.

The three maintenance operations

OPERATION	WHAT IT DOES
Compaction	Merges many small files into fewer large ones
Snapshot expiration	Removes old snapshots, and the data files only they referenced
Unreferenced file removal	Deletes orphaned files no metadata points at

ON THE EXAM

Expiring snapshots is what reclaims storage. It also removes the ability to time travel that far back.

Who runs maintenance, and where

- ◆ **S3 Tables** runs compaction, snapshot expiry and orphan removal automatically.
- ◆ **Glue Data Catalog optimisation** does the same for Iceberg in general purpose buckets.
- ◆ That narrows the gap between managed and self-managed considerably.
- ◆ **Or run it yourself** from a scheduled Spark job on Glue or EMR.
- ◆ Doing nothing is the option that degrades quietly over months.

ON THE EXAM

There are three answers here, not two. Catalog-managed optimisation sits between S3 Tables and writing your own job.

Three ways to compact

- ◆ **Binpack.** The default. Combines small files into larger ones, no reordering.
- ◆ **Sort.** Orders rows by chosen columns, so block statistics can skip more.
- ◆ **Z-order.** Orders across several dimensions at once, for multi-column filters.
- ◆ Available on both S3 Tables and Iceberg tables in general purpose buckets.
- ◆ Sorting costs more to write. It pays off when queries filter on those columns.

ON THE EXAM

This connects directly to module eight. Sorted data makes Parquet's min and max statistics actually useful.

A third kind of bucket, where **AWS runs the maintenance** for you.

How S3 Tables is organised

- ◆ **Table bucket.** The container. Think of it as an analytics warehouse.
- ◆ **Namespace.** A logical grouping of tables within the bucket.
- ◆ **Table.** A structured dataset, always in Apache Iceberg format.
- ◆ **Permissions at table and namespace level**, not only at bucket level.
- ◆ **Integrate with SageMaker Lakehouse** so Athena and Redshift discover them.

ON THE EXAM

Table-level permissions are a real difference. In a general purpose bucket, access control is by prefix.

What S3 Tables charges for

CHARGE	NOTES
Storage	Per GB, as with any S3 storage. Intelligent-Tiering available
Requests	PUT, GET and other API calls against the table
Monitoring	A per-object charge for maintenance monitoring
Compaction	Charged per object processed and per GB processed

ON THE EXAM

You pay for maintenance rather than operating it. Whether that is cheaper depends on how much maintenance the table needs.

S3 Tables or Iceberg in a normal bucket?

S3 TABLES

Managed

- Compaction and snapshot management automatic
- Up to 3x query performance, 10x transactions per second
- Table and namespace level permissions
- Table buckets only, less control over layout

ICEBERG IN A GENERAL PURPOSE BUCKET

Self-managed

- You schedule and run maintenance
- Full control over storage classes and lifecycle
- Works alongside non-table data in the same bucket
- Cheaper if maintenance is genuinely infrequent

THE TIEBREAKER

"No capacity to run table maintenance" is the S3 Tables signal. It is an operational overhead answer.

A retailer streams point of sale transactions into Apache Iceberg tables in a general purpose Amazon S3 bucket, using Amazon Data Firehose. Writes arrive continuously and the tables receive frequent small updates.

Over six months, query times have roughly tripled and storage costs have grown faster than the data volume. The data platform team is three people supporting fifteen other pipelines and has no capacity to build and operate table maintenance jobs.

What should the team do?

- A Migrate the tables to Amazon S3 Tables, which performs compaction and snapshot management automatically
- B Write a scheduled AWS Glue job to run compaction and expire snapshots nightly
- C Increase the Firehose buffer interval so fewer, larger files are written
- D Move the tables to Amazon Redshift so that storage is managed by the warehouse

◆ PAUSE AND DECIDE

What should the team do?

- A

Migrate to S3 Tables
Compaction, snapshot expiry and orphan removal run automatically. Both symptoms addressed with nothing new to operate.
- B

Build a nightly compaction job
Technically correct and exactly what the question ruled out. It adds a job to a team with no capacity to run one.
- C

Larger Firehose buffers
Reduces new small files and does nothing about six months of accumulated ones, or the stale snapshots inflating storage.
- D

Move to Redshift
Replaces the lake to avoid maintaining it. Also loses the open format the tables were chosen for.

THE SIGNAL

Slower queries plus storage growing faster than data is the signature of unmaintained Iceberg tables.

Which engines can read and write Iceberg

- ◆ **Amazon Athena.** Full SQL, including MERGE, UPDATE, DELETE and time travel.
- ◆ **AWS Glue.** Enabled with a job parameter naming the data lake framework.
- ◆ **Amazon EMR.** Spark, Trino and Flink all read and write Iceberg.
- ◆ **Amazon Redshift.** Reads Iceberg tables through the catalog.
- ◆ **Amazon Data Firehose.** Delivers directly into Iceberg tables, including updates.

ON THE EXAM

One table, many engines, no copies. That is the lakehouse claim from module one, made concrete.

Streaming into a table

- ◆ **Amazon Data Firehose** delivers directly into Iceberg tables, including S3 Tables.
- ◆ It can express inserts, updates and deletes, not only appends.
- ◆ **Managed Service for Apache Flink** writes to Iceberg as a sink.
- ◆ **Streaming writes create many small files**, which is why compaction matters most here.
- ◆ **Merge-on-read** is the usual write mode for high-frequency streaming updates.

ON THE EXAM

Streaming plus Iceberg is the combination that most needs maintenance, because it generates small files continuously.

SageMaker Lakehouse, in one slide

- ◆ **Unified access** across S3 data lakes and Amazon Redshift warehouses.
- ◆ Presented through the Iceberg REST catalog, so any Iceberg engine can connect.
- ◆ **Table buckets integrate with it** to become visible to Athena and Redshift.
- ◆ **Permissions come from AWS Lake Formation**, applied once across engines.
- ◆ Part of the SageMaker material added to the exam guide in version 1.1.

ON THE EXAM

The name is new. The idea is the one this course has described since module one: one catalog, many engines.

A data engineer runs a nightly Glue job that rebuilds a curated Iceberg table used by two hundred analysts. Last night the job ran with a faulty join condition and wrote incorrect values across the entire table. The error was discovered at ten this morning.

The raw source data is intact in Amazon S3. Rebuilding the table from raw takes about four hours. Analysts have been querying the incorrect data since the job completed.

What is the fastest way to restore correct data?

- ☐ A Restore the S3 objects from the previous versions using S3 Versioning
- ☐ B Roll the Iceberg table back to the snapshot taken before the faulty job ran
- ☐ C Fix the join condition and rerun the job to rebuild from raw data
- ☐ D Restore the table from the most recent AWS Backup recovery point

◆ PAUSE AND DECIDE

What is the fastest way to restore correct data?

- A

 Restore from S3 object versions
Would mean identifying and reverting hundreds of objects individually, with no guarantee of a consistent set.
- B

 Roll back to the previous snapshot
One metadata operation. No data moves, the table returns to a known consistent state immediately.
- C

 Fix and rerun from raw
Correct, and four hours away. It is what you do after the rollback, not instead of it.
- D

 Restore from AWS Backup
Restores a copy taken at some earlier point, at restore speed, and probably not from immediately before the job.

THE SIGNAL

Rollback is a metadata change, so it is near-instant. That is what makes it a different answer from a restore.

Migrating an existing lake to Iceberg

- ◆ **In-place migration** writes Iceberg metadata over existing Parquet files.
- ◆ No data is rewritten, so it is fast and cheap for a large dataset.
- ◆ **Full migration** rewrites the data, which lets you re-partition and sort at the same time.
- ◆ **Writers must all switch together.** A job still writing plain Parquet bypasses the metadata.
- ◆ **Register the table in the Glue Data Catalog** so every engine sees the same thing.

ON THE EXAM

The fourth point is the real migration risk. A forgotten writer appending files silently makes the table wrong.

An Iceberg table or a Redshift table?

ICEBERG IN THE LAKE

Open and shared

- One copy, read by many engines
- Storage priced as S3
- Good for large, less frequently queried data
- Query latency measured in seconds

AMAZON REDSHIFT

Fast and curated

- Optimised storage, sort and distribution keys
- Sub-second on well-modelled queries
- Costs warehouse capacity
- Best for repeated, concurrent analytics

THE TIEBREAKER

The same rule as module one. Occasional and exploratory favours the lake. Constant and concurrent favours the warehouse.

Where Iceberg is not the answer

- ◆ **Append-only data nobody edits.** Plain Parquet is simpler and needs no maintenance.
- ◆ **Very small datasets.** The metadata overhead outweighs the benefit.
- ◆ **Sub-millisecond key lookups.** That is DynamoDB, as module one established.
- ◆ **Heavy transactional workloads.** A lake table is not an OLTP database.
- ◆ **Adopt it when data changes,** or must be deleted, or is written concurrently.

ON THE EXAM

Iceberg solves problems created by change. Data that never changes does not have those problems.

A logistics company appends GPS ping records to Amazon S3 as Parquet, partitioned by date. Records are never updated or deleted, and the retention policy expires whole partitions after two years. Queries always filter on a date range and read four columns.

A new architect has proposed converting the dataset to Apache Iceberg tables in Amazon S3 Tables, citing ACID transactions and time travel.

How should the data engineer respond?

- ☐ A Agree. Iceberg is the current best practice for data lakes on AWS
- ☐ B Push back. The workload is append-only with whole-partition expiry, so Iceberg adds maintenance without solving a problem the company has
- ☐ C Agree, but use Apache Hudi instead, as it is better suited to append-only data
- ☐ D Agree, and additionally partition by vehicle identifier to improve query performance

◆ PAUSE AND DECIDE

How should the data engineer respond?

- A

 Agree, it is best practice
Best practice is not a requirement. Nothing here updates, deletes rows, or writes concurrently.
- B

 Push back, it solves no problem they have
Append-only, whole-partition expiry, date-range queries. Every Iceberg capability is unused, and maintenance is added.
- C

 Use Hudi instead
Swaps one table format for another. The objection is to the category, not the choice within it.
- D

 Agree and partition by vehicle
Adds the over-partitioning mistake on top. Queries filter by date, which the current layout already serves.

THE SIGNAL

Compare with scenario 1. Same technology, opposite verdict, because the requirement is different.

Symptoms and their causes

SYMPTOM	LIKELY CAUSE
Iceberg queries slowing over months	Small files accumulating. Compaction not running
Storage growing faster than data	Old snapshots retained, or orphaned files never removed
Data written but invisible to queries	A writer bypassing Iceberg and writing files directly
Cannot time travel as far back as expected	Snapshot expiration retention is shorter than assumed
Two concurrent writers, one write lost	Expected. Optimistic concurrency retries or fails the loser
Schema change broke downstream readers	A plain Parquet table, not an Iceberg one

ON THE EXAM

The first two rows are the same root cause seen from two angles: nobody is running maintenance.

Reading an open table format question quickly

- ◆ **Does anything change after it is written?** If not, Iceberg may be unnecessary.
- ◆ **Row-level delete or update?** That is Iceberg, almost by definition.
- ◆ **Who runs maintenance?** Nobody available means S3 Tables.
- ◆ **Recovery from a bad write?** Rollback beats restore, and beats reprocessing.
- ◆ **Named format in the question?** Otherwise Iceberg is the AWS answer.

ON THE EXAM

Next: Amazon Redshift in depth. The largest single module in the course.